ФАЛЬКОН — цифровой признак транспортного средства

Сервис формирует устойчивое векторное представление автомобиля по его визуальным характеристикам и находит тот же автомобиль на снимках с других камер **без использования государственного регистрационного знака**.

Номер на всех кадрах размыт организатором, и ТЗ прямо запрещает опираться на его остаточные признаки под угрозой дисквалификации. Решение работает с формой кузова, цветом, геометрией, дисками, наклейками и повреждениями.

Материалы сдачи

Команда «Агробизнесмены», РГАУ-МСХА имени К.А. Тимирязева. ЛЦТ 2026, задача 7 «Фалькон Тех».

Что	Где
Презентация	docs/presentation.pdf
Видео демонстрации (2 мин 18 с)	релиз demo-v1
Документация	этот README: архитектура (раздел 1), методы (2), запуск (3), сдача (4), метрики и порог (5, 6), источники (7)
Запуск одной командой	`docker compose up --build`, интерфейс на http://localhost:8000, API на /docs
Запуск без видеокарты	`docker compose -f docker-compose.yml -f docker-compose.cpu.yml up --build`
Файлы сдачи для публичного теста	submission/

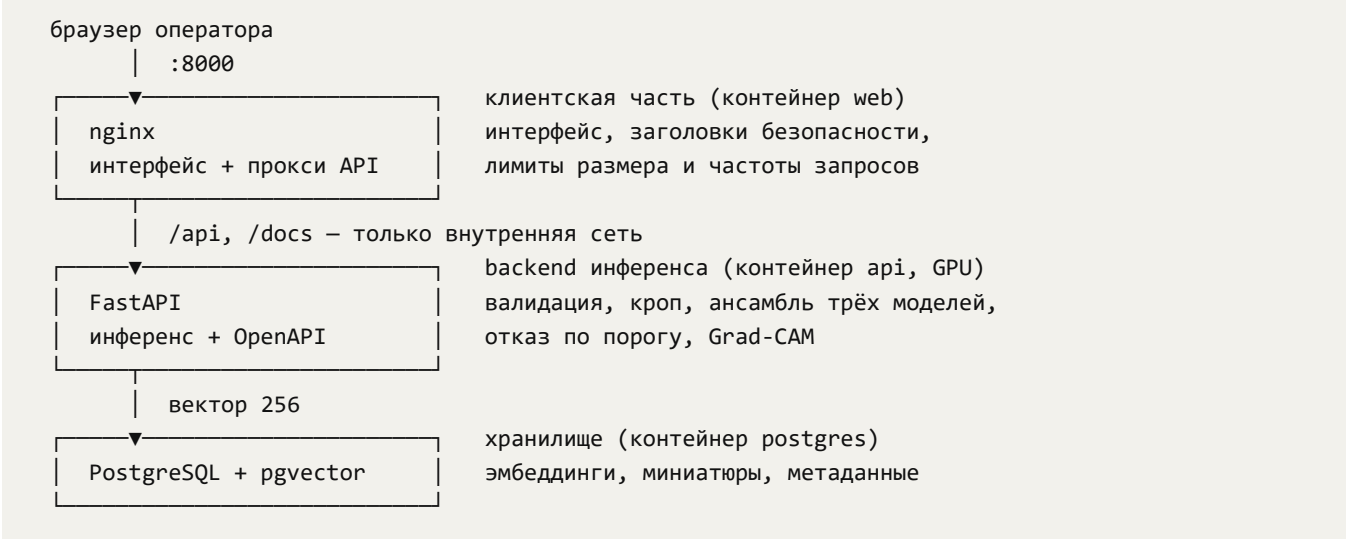
Коротко

Что	Значение	Где проверить
mAP@10 на машинах, которых модели не видели	**0.914** (95% интервал 0.87–0.95) против 0.890 у прежних моделей на тех же машинах	docs/official_check.json, счёт эталонным скриптом организаторов; сравнение — docs/release_choice.json; интервал — docs/confidence_intervals.json
Режим кандидатов 0.7·F1 + 0.3·TNR	**0.930** (F1 0.917, TNR 0.961)	там же, порог 0.69 из docs/threshold_choice.json
Задержка batch=1 / пропускная способность	**28.0 мс / 140 кадр/с**	docs/benchmark_container.json, замер внутри образа
Размер весов	403 МБ при лимите 2 ГБ	три модели в fp16, models/README.md
Воспроизводимость обучения	повтор последнего шага из кода: 0.912 против 0.914, разница −0.002 [−0.008; +0.002]	docs/reproduction_check.json
Опора на номер	нет: закраска номера стоит столько же, сколько контрольная зона	scripts/plate_masking_check.py

Что	Значение	Где проверить
Двойники одной модели и окраски	ответ «требуется проверка»: ловит 10 из 17 ложных совпадений	раздел 6, docs/twin_check.json
Запуск	`docker compose up --build` → http://localhost:8000	раздел 3
Файлы сдачи	submission/ — одна команда `run_submission.py`	раздел 4
Безопасность и закон	журнал обращений, срок хранения, данные не покидают контур заказчика	docs/SECURITY.md, docs/LEGAL.md
Презентация и запись показа	28 слайдов на шаблоне ЛЦТ (обязательные слайды в его оформлении); запись показа 2 мин 18 с: поиск, сравнение, честный отказ, двойники и решение оператора	docs/presentation.pdf, релиз demo-v1

1. Архитектура решения

Компонентная схема



Три роли — три контейнера (раздел 6 ТЗ). Наружу открыт только `web`. Пакетный режим для проверки организаторами работает в обход сервиса: `scripts/run_submission.py` читает CSV и каталог кадров и пишет файлы сдачи.

Функциональные этапы

Этап	Что происходит	Где реализовано
Получение	приём кадра и BBox, валидация, оценка качества	`service/app.py`
Обработка	кроп, препроцессинг, извлечение эмбеддинга	`falcon/extract.py`
Анализ	векторный поиск по галерее, косинусная близость	`service/repository.py`
Результат	Top-10, оценка уверенности либо обоснованный отказ	`falcon/submit.py`

Модель

В сдаче и в сервисе работает **ансамбль из трёх моделей двух архитектур**. Векторы моделей нормируются, склеиваются с равными весами и нормируются снова:

Модель	Архитектура	Предобучение	Вектор	Веса
`clip-ours.pt`	CLIP ViT-B/16 → BNNeck	CLIP (OpenAI)	768	174 МБ, fp16
`clip-veri-ours.pt`	CLIP ViT-B/16 → BNNeck	CLIP, затем VeRi-776	768	174 МБ, fp16
`resnet-v2-veri.pt`	ResNet50-IBN-a → GeM → BNNeck	ImageNet, затем VeRi-776	2048	52 МБ, fp16

Склеенный вектор 3584 проецируется PCA в **256 измерений** (`models/projection.pt`, 3.7 МБ). Главные компоненты посчитаны по обучающей части данных, запросы и галерея проверки в подгонке не участвуют (`scripts/fit_projection.py`). Проекция отбрасывает шумовые и дублирующиеся между моделями направления: mAP@10 растёт на 0.011 (интервал [+0.005; +0.017]), а вектор становится в 14 раз короче — миллион машин занимает 1 ГБ вместо 13.4 ГБ, и pgvector строит индекс HNSW (он работает до 2000 измерений).

Всего **403 МБ весов** при лимите 2 ГБ. Все три модели после основного обучения дообучены с закраской зоны номера (раздел 2), чтобы не опираться на неё. Отражение кадра при выводе (flip TTA) выключено: оно удваивает вычисления, а даёт лишь +0.003 mAP@10.

Почему CLIP. Проба без дообучения на конкурсных данных дала mAP@10 0.110 против 0.061 у ResNet50-IBN-a и 0.055–0.064 у DINOv2: CLIP учился сопоставлять изображения с описаниями и уже различает марки, модели, цвета и детали кузова. После дообучения одна модель CLIP даёт 0.704 против 0.595 у лучшей ResNet. Рецепт как в CLIP-ReID: шаг обучения энкодера 1e-5, в десять раз больше для головы, warmup 5 эпох. ResNet в ансамбле нужна ради разнообразия: свёрточная сеть ошибается иначе, чем трансформер, и добавляет +0.012 mAP@10 (доверительный интервал не содержит нуля). Выбор состава — в разделе 5.

Три решения, которые определяют качество ResNet-ветви:

IBN (Instance-Batch Normalization). Главная трудность задачи в том, что один и тот же автомобиль на разных камерах выглядит по-разному: другое освещение, баланс белого, время суток, погода. InstanceNorm нормирует статистики каждого снимка по отдельности и вычищает «стиль» кадра, сохраняя содержание. IBN-a ставит половину каналов под IN, половину под BN в первых трёх стадиях: низкие слои избавляются от стиля, высокие сохраняют различающую информацию.

BNNeck. Triplet loss работает в евклидовом пространстве, а классификация идентичностей — с косинусными углами. Считать обе по одному вектору значит получить конфликтующие градиенты. BNNeck разводит их: triplet до BatchNorm, классификация после, поиск — по вектору после BN.

last_stride = 1. Прореживание в последней стадии убрано, карта признаков 16×16 вместо 8×8. Признаки, которые задача просит различать — диски, наклейки, повреждения — мелкие, терять по ним разрешение нельзя.

Вход 256×256. Пешеходный ReID использует вытянутые 128×256, но автомобиль в кадре ближе к квадрату, и вытягивание искажает пропорции кузова — один из основных различающих признаков.

---

2. Методы обработки данных и алгоритмы

Обучение

Рецепт основан на «Bag of Tricks for Person Re-ID» (Luo et al., 2019): ID-loss со сглаживанием меток 0.1 + batch-hard triplet + center loss с малым весом, warmup 10 эпох, косинусное затухание скорости обучения, AMP fp16.

CLIP ViT-B/16 учится по тому же рецепту и на тех же PK-батчах, но с раздельным шагом: 1e-5 для энкодера и 1e-4 для головы (`--backbone-lr`), weight decay 1e-4, warmup 5 эпох, 60 эпох. Вторая модель CLIP сначала 12 эпох учится на VeRi-776, затем дообучается на конкурсных данных. Весь прогон воспроизводится одной командой: `python scripts/overnight_clip.py <каталог данных>`.

Закраска зоны номера при обучении. Ответ 48 организаторов: финалистов проверяют на тесте, где зона номера закрашена сплошной заливкой. Номер размыт, но размытое пятно и артефакты его границ остаются признаком, за который модель может зацепиться. Аугментация `PlateZoneErase` закрашивает зону внизу по центру кропа случайным сплошным цветом; центр и размер зоны случайны, потому что номер висит по-разному у разных машин и ракурсов. Каждая модель сдачи дообучена с ней 15 эпох от своего лучшего чекпойнта (`scripts/finetune_plate_erase.py`): опора на зону номера ушла, а mAP@10 ансамбля даже выросла на +0.006 (интервал [+0.0002; +0.012]).

PK-сэмплирование с кросс-камерным приоритетом. Batch-hard triplet ищет самый трудный позитив и негатив внутри батча, поэтому случайный батч бесполезен: в нём нет пар одного ТС. Батч собирается как P идентичностей по K снимков, и снимки одной машины берутся с *разных* камер. Это не косметика: в метрике засчитываются только кросс-камерные совпадения, и учить модель надо ровно им.

Аугментации отвечают конкретным пунктам раздела 5 ТЗ про покрытие условий: ColorJitter — время суток и погода; RandomErasing — частичное перекрытие; Pad + RandomCrop — неточность рамки BBox; горизонтальное отражение — ракурсы; PlateZoneErase — закраска зоны номера (раздел 9 ТЗ, ответ 48). Вертикальное отражение и сильный поворот не используются: камеры городской инфраструктуры не переворачивают кадр.

Поиск

Косинусная близость по L2-нормированным векторам 256. В сервисе — pgvector с индексом HNSW (`vector_cosine_ops`); в пакетном режиме на конкурсной галерее из 750 объектов используется точный перебор.

Это решение подкреплено измерением, а не предположением: на галерее 10 000 объектов приближённый поиск втрое **медленнее** точного и при этом теряет соседей, точка перехода лежит около 100 000 объектов, а на 10^6 HNSW даёт ускорение ×7.5 при Recall@10 = 0.95. Полные замеры — docs/SCALABILITY.md.

Обогащение векторов соседями. Перед ранжированием вектор объекта галереи усредняется по сходству со своими ближайшими соседями по галерее (DBA, k=3), а вектор запроса — со своими ближайшими объектами уже обогащённой галереи (alpha-QE, k=2). Один снимок видит машину с одной точки, среднее нескольких снимков описывает саму машину устойчивее. Сосед подмешивается, только если сходство с ним не ниже 0.4: у машины с единственным снимком в галерее вектор остаётся прежним. Вес соседа — его сходство **в кубе**: близкий сосед влияет почти как сам снимок, дальний

почти не влияет. Прирост без какого-либо переобучения: mAP@10 0.7565 → 0.7957 (бутстреп +0.040, 95% интервал [+0.027; +0.053]).

Степень веса выбрана по размеру галереи, а не на глаз. У нас на 890 запросов 1541 снимок галереи, в публичном тесте на 1110 запросов — 750, поэтому каждая конфигурация проверена на галереях четырёх размеров (по 12 прогонов на размер: 3 прореживания × 4 жеребьёвки):

Вес соседа	**500**	**750 (как в тесте)**	**1100**	**1541 (наша)**
сходство	0.8070	0.8042	0.7946	0.7959
сходство^2	0.8183	0.8126	0.8014	0.7974
сходство^3	**0.8199**	**0.8163**	**0.8021**	0.7971

На нашей плотной галерее разница в пределах шума, а на плотности закрытого теста куб даёт +0.012 mAP@10. Ни на одном размере он не проигрывает, поэтому в сдаче он. Устройство самого публичного теста подтверждает выбор: медианное сходство галерейного снимка с ближайшим соседом там 0.64, со вторым 0.50, с третьим 0.43 — то есть у снимка обычно есть один настоящий «свой» сосед, а дальше идут чужие машины. Линейный вес подмешал бы их с силой 0.50 и 0.43 против 0.64 у своего, куб — 0.13 и 0.08 против 0.26. Именно поэтому на редкой галерее куб выигрывает, а не просто «сглаживает». Число соседей при этом остаётся прежним: на плотной галерее выгодно подмешивать больше (DBA 3 + QE 4 давало +0.007), но на галерее размера теста то же изменение теряет 0.024 — выигрыш оказался свойством нашей плотной галереи, а не модели, и в сдачу не взят.

Правило ответа 38 соблюдено: связей «запрос — запрос» нет, запрос обогащается только векторами галереи, галерея — своими же. Выдача каждого запроса не зависит от того, какие ещё запросы обрабатываются, и это закреплено тестом (`tests/test_expansion.py`). Уверенность для режима отказа считается по ИСХОДНОМУ косинусу: обогащение смещает сходства вверх, а шкала порога должна остаться прежней.

k-reciprocal re-ranking реализован в `falcon/rerank.py` в варианте, совместимом с потоковым протоколом: используется только близость «запрос → галерея» и «галерея → галерея». Классический алгоритм строит совместную матрицу по объединению всех запросов и галереи и опирается в том числе на связи «запрос — запрос», что ответ 38 организаторов прямо запрещает. Независимость выдачи от состава остальных запросов закреплена тестом.

По умолчанию re-ranking **выключен**: на короткой галерее он ухудшал метрику (проверено и на текущем ансамбле: 0.71–0.76 против 0.796 без него), и включать его следует только после A/B-проверки на локальном сплите.

Оценка качества входа

Базовая валидация по разделу 4 ТЗ: размер кропа, резкость по дисперсии градиента, яркость. Предупреждения показываются оператору, но не блокируют обработку — решение о пригодности кадра остаётся за человеком.

Интерфейс оператора

`http://localhost:8000` — рабочее место оператора. Это тонкий клиент без внешних библиотек и CDN, он работает без интернета. Шрифты (Oswald, Golos Text, JetBrains Mono, лицензия SIL OFL) лежат в `service/static/fonts`.

Идея оформления — **«Номер можно размыть. Отпечаток — нет.»** Номер машины размыт, зато у неё есть отпечаток: вектор, который модель считает по кузову. Интерфейс рисует его папиллярным узором, как отпечаток пальца, и из этого выросло всё остальное: бумага и чернила дактилоскопической карты, синий номерного знака. Синий значит «совпадение», красный — «не совпало», янтарный — «нужна проверка».

- **Схема города** (полоса сверху). По улицам едут машины, камеры обводят их рамкой, одну машину узнают на разных камерах, и маршрут тянется между ними. Схема реагирует на работу: при загрузке кадра на ней появляется цель, во время поиска по камерам идёт волна, после ответа у камер всплывают настоящие снимки найденных кандидатов в табличках: синие — выше порога, чёрные — ниже. Сутки на схеме идут ускоренно: днём город — белый бумажный макет, ночью — чертёж-синька с горящими окнами; время показывают часы в углу. Подписей на схеме нет намеренно — камера, машина и цель узнаются по рисунку, а номера камер и сходство видны там, где они нужны: в карточках результата. Камеры на схеме условные: их расположения нет в данных по условию задачи.
- **Запрос.** Кадр перетаскивается в окно, вставляется из буфера (Ctrl+V) или выбирается файлом. Рамка машины рисуется мышью прямо на кадре, координаты можно поправить вручную.
- **Результат.** Вердикт («Отпечаток совпал» или обоснованный отказ «Отпечаток не найден»), десять кандидатов с миниатюрами, сходством и узором-отпечатком, наложенным на отпечаток запроса: синий — линии совпали, красный — разошлись. Выгрузка в CSV и JSON (раздел 10 ТЗ: «экспортировать результаты»).
- **Сравнение.** По клику на кандидата открывается окно «запрос ↔ кандидат»: отпечатки запроса и кандидата и их наложение, карта внимания Grad-CAM по обеим архитектурам ансамбля и проверка номера маскированием (см. раздел 5, «Проверка на опору на номер»).
- **Решение оператора.** В окне сравнения — «Это она» и «Не она». Решение сохраняется в базе (`POST /api/feedback`) без кадров и векторов: какой снимок галереи показан, что решил человек, какое сходство он видел и что ответил сервис. Сводка — `GET /api/feedback`, выгрузка для перекалибровки порога и зазора двойников на настоящих решениях — `GET /api/feedback/export.csv`. Подтверждённый кадр можно одной кнопкой добавить в галерею этой машины: это новый ракурс, и в следующий раз машину найдут и с него. Двойников модель не различает, человек — различает, и его решения становятся данными.
- **Галерея.** Наполняется по одному кадру или пачкой: 20 снимков за 0.6 с.

Отпечаток — проекция эмбеддинга на 128 фиксированных случайных направлений (`fingerprint()` в `service/app.py`). Такая проекция приближённо сохраняет сходство (лемма Джонсона — Линденштраусса): у снимков одной машины числа отпечатка коррелируют на 0.97, у похожей чужой машины — на 0.53–0.64. Узор строится из этих чисел: гребни вокруг ядра изогнуты гармониками, коэффициенты которых — числа отпечатка, поэтому близкие векторы дают близкие узоры. На 120 снимках 30 машин ближайший отпечаток принадлежит той же машине в 97% случаев. В поиске отпечаток не участвует, это объяснение для оператора.

Вся работа сервиса с видеокартой идёт в одном выделенном потоке, прогретом при старте. Кэш автоподбора алгоритмов cuDNN у PyTorch свой в каждом потоке, и поиск, попавший в свежий поток пула FastAPI, раньше занимал 5.8–8.8 с вместо 0.1 с.

---

3. Сборка и запуск

Получение кода

Веса моделей (`models/*.pt`, 399 МБ на три файла) хранятся в Git LFS: обычный git не принимает файлы больше 100 МБ. Клонировать нужно с установленным Git LFS:

```
git lfs install
git clone https://github.com/ski11s100/falcon-reid.git
```

Архив ZIP со страницы GitHub содержит вместо весов текстовые указатели LFS. Сервис это распознаёт и сообщает, как докачать веса: `git lfs pull`. Если у репозитория исчерпана месячная квота трафика Git LFS (у бесплатного GitHub она небольшая, а один клон — это 403 МБ весов), веса докачивает скрипт на стандартной библиотеке Python — из релиза GitHub, со сверкой SHA-256 с хешем из самого указателя:

```
python scripts/fetch_weights.py
```

Всё решение одной командой

```
docker compose up --build
```

Поднимаются три контейнера: `web` (интерфейс и прокси), `api` (инференс на GPU) и `postgres` (pgvector). Интерфейс оператора — `http://localhost:8000`, спецификация OpenAPI — `http://localhost:8000/docs` (Swagger UI собран из локальных файлов и открывается без интернета).

Для GPU нужен установленный NVIDIA Container Toolkit. Сборка образа требует интернета, запуск — нет: веса модели лежат внутри образа, во время инференса ничего не скачивается.

Без видеокарты NVIDIA — например, на обычном облачном сервере для работающего прототипа — то же решение поднимается на процессоре:

```
docker compose -f docker-compose.yml -f docker-compose.cpu.yml up --build
```

Веса, порог и интерфейс те же, ответы совпадают с GPU до третьего знака. Поиск занимает около 0.4 с на процессоре ноутбука вместо ~30 мс на видеокарте, поэтому скорость так не меряется.

Пароль базы и ключ API задаются переменными (`cp .env.example .env`). Без `.env` решение тоже запускается, со значениями для локального показа. Меры защиты описаны в docs/SECURITY.md. Каждый поиск, регистрация и удаление пишутся в журнал обращений (`/opt/falcon/data/audit.jsonl`): кто, когда и что искал — без кадров и векторов. Срок хранения галереи задаётся `FALCON_RETENTION_DAYS`. Правовая сторона — персональные данные, меры защиты по приказу ФСТЭК №21 и лицензии моделей и данных — в docs/LEGAL.md.

Демонстрационная галерея — 12 машин по 4–6 снимков с разных камер из `train.csv` и пара двойников для показа ответа «требуется проверка» (раздел 6), всего 69 снимков — заполняется одной командой через API сервиса:

```
python scripts/seed_demo_gallery.py /путь/к/данным --url http://localhost:8000
```

Векторы считает сам сервис, поэтому при смене модели галерею достаточно пересоздать: `docker compose down -v`, затем `up` и эта команда. Размерность вектора зашита в схему базы, и сервис со старой базой не запустится, а подскажет, что делать.

Генерация файлов сдачи

```
docker run --rm --gpus all -v /путь/к/данным:/data falcon-api \
    python scripts/run_submission.py /data --output /data/submission
```

Входной каталог должен содержать `test_query.csv`, `test_gallery.csv` и `images/`. На выходе: `submission.csv`, `embeddings.npy`, `candidates.csv`, `manifest.json`. Порог отказа по умолчанию — откалиброванный 0.69 (`--threshold` его меняет). Скрипт сам проверяет формат результата и завершается с ошибкой при расхождении.

Готовые файлы сдачи для публичного теста лежат в submission/ (раздел 8 ТЗ): они получены именно этой командой в собранном образе, запущенном без сети (`--network none`), за 39 с; замер производительности — отдельный запуск с `--benchmark` (docs/benchmark_container.json). Повторный запуск на Windows даёт тот же первый кандидат во всех 1110 запросах; порядок почти равных кандидатов может отличаться из-за вычислений fp16 на разных системах.

Локальная разработка без Docker

```
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m pip install torch==2.14.0 torchvision==0.29.0 `
    --index-url https://download.pytorch.org/whl/cu126
$env:FALCON_CHECKPOINTS = "models/clip-ours.pt,models/clip-veri-ours.pt,models/resnet-v2-veri.pt"
$env:FALCON_MATCH_THRESHOLD = "0.69"
$env:FALCON_TWIN_MARGIN = "0.05"
$env:FALCON_PROJECTION = "models/projection.pt"
.\.venv\Scripts\python.exe -m uvicorn service.app:app --port 8000
```

Без `FALCON_DATABASE_DSN` сервис использует локальный SQLite — удобно для разработки, в docker-compose всегда подключён PostgreSQL.

Полный цикл одной командой

От сырого датасета до готовых файлов сдачи:

```
.\.venv\Scripts\python.exe scripts/run_pipeline.py D:/falcon-data --output runs/v1
```

Последовательно выполняются: аудит датасета, обучение с валидацией по официальной метрике, калибровка порога отказа, A/B-проверка re-ranking, проверка на остаточные признаки номера, генерация сдачи и замер производительности. Каждый этап пишет свой отчёт в каталог результата.

Размер батча подбирается под доступную видеопамять автоматически. На Windows это существенно: при нехватке видеопамяти драйвер не бросает ошибку, а начинает вытеснять тензоры в системную память, и обучение молча замедляется в разы.

Только обучение

```
.\.venv\Scripts\python.exe falcon/train.py <каталог данных> --output runs/v1 --epochs 60
```

Тесты и линтер

```
.\.venv\Scripts\python.exe -m unittest discover -s tests
.\.venv\Scripts\python.exe -m ruff check .
```

153 теста и линтер проходят на каждом пуше в GitHub Actions (.github/workflows/tests.yml) вместе с `pip-audit`: на чистой машине ставятся зафиксированные версии, чтобы решение точно собиралось не только у нас. Набор правил линтера и причины отключений — ruff.toml.

---

4. Формат сдачи

Формат подтверждён ответами организаторов на вопросы участников и закреплён тестами в `tests/test_submission_format.py`.

Файл	Правило
`embeddings.npy`	float32, `(len(query) + len(gallery), D)`. Сначала **все** query в порядке строк файла, затем **вся** gallery. Без дополнительной сортировки
`submission.csv`	**Без заголовка**, как читает эталонный скрипт: `query_id` и ровно **10** `gallery_id`. Для каждого запроса без исключений, включая те, где мы уверены в отказе. Отказ в этом файле не выражается
`candidates.csv`	`query_id,gallery_id,confidence`. **Отказ = полное отсутствие строк** для этого `query_id`. Не пустая строка, не специальное значение

`confidence` — оценка близости, не калиброванная вероятность. Диапазон произвольный, требуется только монотонность по уверенности.

На принятый запрос пишется ровно одна строка — лучший кандидат. Эталонный скрипт принимает решение по кандидату с наибольшей `confidence` и лишние строки в F1 не засчитывает, а у нас порядок кандидатов определяют обогащённые векторы, а уверенность — исходный косинус. Вторая и третья строки не добавили бы ничего, но могли бы подменить наш первый кандидат другим, у которого исходный косинус случайно выше.

---

5. Протокол оценки и метрики

Официальный протокол реализован в `falcon/metrics.py` и покрыт 19 тестами. Он отличается от «обычного» ReID-протокола, и это принципиально:

- **mAP@10** считается по `submission.csv`, не по полному ранжированию;
- **junk-фильтр** убирает из галереи объекты, у которых совпадают *одновременно* `vehicle_id` и `camera_id`. Объекты той же камеры с другим ТС остаются — это валидные и самые сложные негативы;
- фильтр применяется **до** усечения до десяти позиций;

- **AP нормируется на** `min(n_pos, 10)` — иначе ТС, снятое восемью камерами, не могло бы получить AP = 1 даже при идеальной выдаче;
- запросы без валидного позитива **исключаются** из mAP, а не получают AP = 0.

Конкурсные данные

Параметр	Значение
Кадров в `images/`	11 416, полные кадры 1920×1080, JPEG
`train.csv`	9 556 наблюдений, 1 541 автомобиль, 96 камер
`test_query.csv` / `test_gallery.csv`	1 110 / 750 объектов
Снимков на автомобиль	от 4 до 8, в среднем 6.2
Снято двумя и более камерами	1 541 из 1 541, то есть 100%
Меньшая сторона кропа по BBox	минимум 173, медиана 376, максимум 511 пикс.
Доля площади кадра под рамкой	медиана 15.9%

Съёмка дворовая и парковочная, ракурс преимущественно сверху. Номера и лица размыты организатором. Проверка целостности — `scripts/check_data.py`.

Важное следствие для препроцессинга: кроп всегда вырезается из полного кадра 1920×1080, поэтому декодирование JPEG занимает заметную долю замеряемого цикла и оптимизируется через `draft()` (см. `falcon/data.load_crop`).

Локальный сплит

`falcon/data.py` строит из `train.csv` аналог закрытого теста: ID обучения и валидации не пересекаются, у каждого закрытого запроса гарантирован кросс-камерный позитив, и около **20% запросов не имеют пары в галерее** — та же пропорция, что в закрытом тесте. Без такого сплита порог отказа калибровать не на чем.

Результаты

Модель — ансамбль двух CLIP ViT-B/16 и ResNet50-IBN-a (раздел 1), дообученных с закраской зоны номера, с PCA-проекцией в 256 измерений, без отражения кадра, и затем дообученных ещё раз на 1445 машинах вместо 1156 (ниже, «Последний шаг»).

Проверка — на 96 машинах из отложенной части, которых не видела ни одна версия моделей: 226 запросов, из них 51 без пары в галерее (22.6%), галерея 377 снимков. Галерея здесь вчетверо меньше прежней (1541 снимок на 385 машин), поэтому абсолютные числа выше, чем на прежней проверке, и сравнивать версии можно только на одной и той же выборке — так и сделано.

Все числа ниже пересчитаны **эталонным скриптом организаторов** (`organizers/evaluate.py`) и совпадают с нашим `falcon/metrics.py` до последнего знака — это проверяет `tests/test_official_equivalence.py`, а на реальной отложенной выборке — `scripts/verify_official.py` (отчёт docs/official_check.json).

Метрика	Прежние модели (1156 машин)	Сдача (1445 машин)
mAP@10	0.8899	**0.9138**

Метрика	Прежние модели (1156 машин)	Сдача (1445 машин)
Rank-1	0.8914	0.9086
Rank-5	—	0.9657
Режим кандидатов 0.7·F1 + 0.3·TNR, свой порог	0.915 (F1 0.912, TNR 0.922, порог 0.525)	**0.930** (F1 0.917, TNR 0.961, порог 0.69)
mAP по полному ранжированию	—	0.9211
mINP	—	0.9100
PR-AUC (по эталонному скрипту)	—	0.9785

Прежняя проверка — 385 машин, галерея 1541 снимок — для новых моделей больше не годится: три четверти этих машин ушли в обучение. На ней прежние модели давали mAP@10 0.7956 и балл кандидатов 0.868 при пороге 0.605; эти числа остаются точкой отсчёта для истории выбора состава ниже.

Выбор состава: точность, значимость и скорость вместе

Кандидаты сравнивались на одном сплите с настройками сдачи (`scripts/compare_ensembles.py`, отчёт docs/ensemble_choice.json). Порог отказа калибровался для каждого варианта отдельно, скорость мерилась по методике организаторов на RTX 3060 Laptop:

Вариант	mAP@10	Кандидаты	FPS
ResNet с нуля	0.5642	—	—
ResNet поверх VeRi-776	0.5946	—	—
две ResNet (прежняя сдача)	0.6119	0.840	147
CLIP	0.7044	0.850	200
CLIP + ResNet	0.7231	0.858	119
2×CLIP	0.7324	0.867	101
2×CLIP + ResNet, с отражением	0.7419	0.873	**75**
2×CLIP + ResNet, без отражения	0.7393	0.875	143
то же, дообучено с закраской номера	0.7455	0.873	126
то же + PCA-проекция в 256	0.7565	0.872	131
то же + обогащение векторов соседями (сдача)	**0.7956**	**0.868**	**131**

Парный бутстреп по запросам (5000 выборок):

- сдача против прежних двух ResNet: **+0.144**, 95% интервал [+0.127; +0.163];
- вклад ResNet в ансамбль (против 2×CLIP без отражения): +0.012, интервал [+0.004; +0.020] — не шум, свёрточная сеть ошибается иначе;
- цена отказа от отражения кадра: −0.003, интервал [−0.007; +0.002] — неотличимо от нуля, а вычислений вдвое меньше;
- дообучение с закраской зоны номера: +0.006, интервал [+0.0002; +0.012];
- обогащение векторов соседями (DBA + alpha-QE с кубическим весом): **+0.039**, интервал [+0.028; +0.050]; приём работает поверх готовых векторов, ничего не обучая, и устойчив к параметрам: вся сетка DBA 1–5 × QE 0–3 на галерее размера теста даёт 0.784–0.813;

- PCA-проекция в 256 измерений: +0.011, интервал [+0.005; +0.017]. Проекция помогает на обеих плотностях галереи (средний mAP@10 0.8061 против 0.7885 без неё), а размерность лежит на плато: 128 даёт 0.8073, 192 — 0.8066, 256 — 0.8061, 384 — 0.8012, 512 — 0.7985. Разница между 128 и 256 (0.001) меньше разброса между жеребьёвками, поэтому взят запас: 256 измерений вчетверо меньше исходных 3584, помещаются в индекс pgvector и оставляют место под другое распределение закрытого теста. Отбеливание (деление на корень собственного значения) проверено и отвергнуто: −0.02…−0.08.

Последний шаг: больше машин в обучении

Все приёмы выше подбирались на отложенной выборке из 385 машин, и модели видели при обучении только 1156 машин из 1541. Когда рецепт был зафиксирован, три четверти отложенных машин (289) отданы в обучение: `falcon/train.py --extra-train-share 0.75`, те же 15 эпох и скорости обучения, что при дообучении с закраской номера. Оставшиеся 96 машин не видела ни одна версия, и на них обе версии сравниваются напрямую (`scripts/compare_release.py`, docs/release_choice.json):

Модель	Прежняя (1156 машин)	Новая (1445 машин)
clip-ours	0.8418	0.8822
clip-veri-ours	0.8271	0.8655
resnet-v2-veri	0.7412	0.7926
ансамбль с проекцией и обогащением	0.8899	**0.9138**

Прирост ансамбля +0.024, парный бутстрэп даёт 95% интервал [+0.008; +0.042] — ноль в него не попадает. Этот шаг повторён заново из кода репозитория — те же стартовые веса, данные, эпохи и seed: 0.912 против 0.914, разница −0.002 с интервалом [−0.008; +0.002], то есть обучение воспроизводится (docs/reproduction_check.json). Каждая модель взята с последней, 15-й эпохи, а не с лучшей по этим 96 машинам: иначе выбор эпохи подглядывал бы в проверку и завышал результат. Балл режима кандидатов тоже вырос — 0.915 → 0.930, у каждой версии свой порог, посчитанный на тех же 96 машинах. Задержка 29 мс и 141 кадр/с — в пределах лимитов.

Новые модели увереннее, и шкала сходства у них выше: при прежнем пороге 0.605 они принимали бы машины без пары за совпадение (TNR 0.73). Порог для них посчитан заново тем же способом — жеребьёвки и две плотности галереи на 96 машинах — и равен 0.69 (раздел 6).

Следующий шаг напрашивался сам — отдать в обучение и последние 96 машин. Тот же рецепт на всех 1541 машинах обучен (`--extra-train-share 1.0`, `--resume-dir` с весами до дообучения), но проверять его уже не на чем. Поэтому правило решения записано до обучения, а проверка идёт сверкой сдач на публичном тесте (`scripts/compare_submissions.py`, docs/fulldata_check.json). Первый кандидат совпал с нынешней сдачей в 96.3% запросов (порог правила — 94%, столько совпадали версии на 1156 и 1445 машинах), отказов 3.7% против 3.5%, шкала уверенности та же. Но у двух моделей из трёх точность на обучении упала на 0.033–0.034 при допуске 0.03. Правило не выполнено, и сдача не меняется: у моделей на 1445 машинах точность измерена на машинах, которых они не видели, а у версии на всех машинах её измерить нельзя. Допуск задан с запасом лишь в 0.003, но менять правило после того, как увидел результат, — это подгонка.

Дообучение всего ансамбля на входе 288×288 вместо 256×256 (ночной прогон `scripts/overnight_resolution.py`, свои PCA-проекция и порог) проверено и отклонено скриптом `scripts/compare_release.py` (docs/release_choice_288.json): mAP@10 0.7930 против 0.7957, разница

−0.0027 с интервалом [−0.013; +0.009]; балл кандидатов 0.856 против 0.868; задержка 42.3 мс — выше лимита 40 мс. Крупнее вход — не лучше: кузов по-прежнему сжимается в квадрат, а вычислений на 27% больше.

Вторая попытка — вход 224×320 с пропорциями кузова вместо квадрата (позиционные эмбеддинги ViT перенесены на сетку 14×20 патчей, 30 эпох на модель) — тоже отклонена (docs/release_choice_224x320.json): mAP@10 0.7831 против 0.7957, разница −0.0126 с интервалом [−0.024; −0.001], то есть хуже значимо. Модели, обученные на квадрате, не выигрывают от смены геометрии за 30 эпох дообучения: они теряют больше, чем получают от лишних деталей по ширине.

С отражением самый точный вариант выпадает из полного балла за пропускную способность (75 FPS при пороге 100). Без отражения он даёт 126–143 FPS и 31–37 мс на кадр (Windows и контейнер) при порогах 100 FPS и 40 мс — столько же, сколько прежняя сдача из двух ResNet.

Проверка на опору на номер

Ответ 48 организаторов: финалистов проверяют на тесте, где зона номера закрашена сплошной заливкой. `scripts/plate_masking_check.py` повторяет это на локальном сплите: закрашивает зону на всех снимках запросов и галереи и пересчитывает mAP@10. Для сравнения — контрольные зоны той же площади (docs/plate_masking_check.json):

Сдача проверена на 96 машинах, которых модели не видели (базовый mAP@10 без обогащения — 0.8976); прежние версии — на прежней проверке из 385 машин:

Закрашенная зона	Сдача (1445 машин)	Прежняя сдача	До дообучения с закраской
номер (узко)	**−0.005**	−0.017	−0.019
рамка номера (широко)	**−0.006**	−0.023	—
верх кузова	−0.025	−0.009	—
низ справа	−0.013	−0.009	−0.009
правый бок	−0.010	−0.008	—
левый бок	−0.008	−0.017	—
низ слева	−0.004	−0.017	−0.017
решётка над номером	−0.003	−0.008	−0.005

У сдачи закраска номера стоит меньше, чем закраска большинства участков кузова той же площади: верх кузова, боковины и правый низ важнее для модели, чем зона номера. До дообучения с `PlateZoneErase` закраска номера стоила чуть больше худшей контрольной зоны; после него и после добавления машин опора на номер исчезла. Рамка в проверке к тому же больше самого номера и задевает крепление и бампер, так что при закраске одного номера падение будет ещё меньше.

Режим кандидатов: правило подтверждено эталонным скриптом

До выхода эталонного скрипта было неясно, засчитывается ли верхний кандидат, если это та же машина **с той же камеры**, что и запрос: у 44% наших запросов ближайший кандидат именно такой, и от толкования зависело 3.7 балла из 10. Эталонный скрипт (`candidate_metrics`) ответил: верность проверяется только по `vehicle_id`, камера не важна. Наш `falcon/metrics.py` приведён к этому правилу, а порог отказа подобран заново под настоящую метрику (раздел 6).

Переранжирование: не используется, прирост неотличим от шума

k-reciprocal re-ranking реализован в потоково-совместимом варианте и проверен на реалистичном локальном сплите (890 запросов, галерея 1541 снимок) для прежнего ансамбля из двух ResNet:

Конфигурация	mAP@10
без re-ranking	**0.6119**
k1=12, λ=0.7	0.6188
k1=12, λ=0.5	0.5946
k1=20, λ=0.7	0.5858
k1=12, λ=0.3	0.5686
k1=20, λ=0.3	0.4875

Лучшая конфигурация даёт +0.6–0.7 пункта, но это максимум из шести проб, то есть оценка завышена выбором. Парный бутстрэп по запросам (10 000 выборок) для неё: разница +0.0057, **95% интервал [−0.010; +0.022]**, запросов стало лучше 208, хуже 197. На предыдущем сплите та же конфигурация теряла 4.7%. Выигрыш не доказан, потеря возможна, поэтому в сдаче метод выключен. Для ручного включения (`--rerank`) по умолчанию взяты лучшие параметры сетки.

Производительность

Замер по методике организаторов: медиана полного цикла при batch=1 (чтение файла, декодирование, кроп, препроцессинг, forward, L2-нормализация и проекция), 300 прогонов после 50 прогревочных, с синхронизацией CUDA. Измерено в собранном образе, запущенном без сети (`--network none`), на RTX 3060 Laptop (Ampere SM86, та же архитектура, что RTX A5000 на стенде жюри, но вдвое слабее) на реальных конкурсных кадрах 1920×1080 (docs/benchmark_container.json):

Показатель	Значение	Порог полного балла	Балл
Latency batch=1 (медиана)	**28.0 мс**	≤ 40 мс	10 / 10
Latency batch=1 (p95)	32.0 мс	—	справочно
Throughput (лучший, batch 32)	**140 FPS**	≥ 100 FPS	10 / 10
Throughput при batch=1	88 FPS	—	справочно
Пиковая видеопамять	1.6 ГБ из 24	—	справочно
Суммарный размер весов	403 МБ	≤ 2 ГБ	допущено

Итого 20 из 20 на видеокарте вдвое слабее стенда жюри.

CUDA Graphs. Разбор полного цикла показал, куда уходит время: чтение и декодирование кадра 9 мс, препроцессинг 2 мс, три модели 20 мс, причём ResNet при batch=1 шла 10 мс — дольше обеих CLIP. Сотни мелких ядер упирались не в видеокарту, а в процессор, который их запускает. Для batch=1 и 16 прямой проход записывается в CUDA Graph один раз и дальше воспроизводится одной командой (`FeatureExtractor._replay`): ResNet 10.3 → 2.6 мс, все модели 20 → 11.5 мс, полный цикл в образе 36.8 → 29.5 мс. Векторы при этом те же (косинус с обычным проходом 0.9999999). Это запас именно для стенда жюри: у Xeon Gold 6338 на 2.0 ГГц запуск ядер медленнее, чем у ноутбучного процессора, на котором мы меряем.

Три модели укладываются в бюджет и потому, что CLIP ViT-B/16 в fp16 на тензорных ядрах почти так же быстр, как ResNet50, а отражение кадра выключено: оно удваивало вычисления ради +0.003 mAP@10.

Соответствие ограничениям конкурса

Требование	Где задано	Решение
Веса ≤ 2 ГБ, иначе решение не допускается к оценке производительности	ответ 34	403 МБ: две CLIP в fp16, ResNet в fp16, проекция
Latency batch=1 ≤ 40 мс, полный цикл с чтением кадра	ответы 31, 34	28.0 мс медиана, 32.0 мс p95 на карте вдвое слабее A5000
Throughput ≥ 100 FPS	ответ 34	140 FPS
Инференс без интернета, веса в составе решения	раздел 8 ТЗ, ответ 39	образ запускается с `--network none`, `HF_HUB_OFFLINE=1`, модели строятся без загрузки весов
Запуск одной командой	разделы 6, 8, 12 ТЗ	`docker compose up`; сдача — одна команда `docker run … run_submission.py`
`submission.csv`: без заголовка, ровно 10 кандидатов	раздел 8 ТЗ, эталонный скрипт	проверяется `validate_submission` и загрузчиками эталонного скрипта
Отказ — отсутствие строк в `candidates.csv`	эталонный скрипт	так и пишется; 39 отказов из 1110 на публичном тесте
Порядок `embeddings.npy`: все query, затем вся gallery	ответ организаторов, эталонный скрипт	проверяется тестом формата
`camera_id` на инференсе не используется	ответ 29	признаки считаются только по кропу; `camera_id` нужен лишь для обучения и локального сплита
TTA только в пределах одного изображения, без связей «запрос — запрос»	ответ 38	отражение кадра выключено, re-ranking выключен, проекция подогнана на обучающей части, а не на тесте
Внешние веса и данные — публичные и перечислены	раздел 7 ТЗ, ответ 46	CLIP (OpenAI), IBN-Net, VeRi-776 — раздел 7
Нельзя опираться на номер	разделы 5, 9 ТЗ, ответ 48	дообучение с закраской зоны номера и проверка закраской (раздел 5)

---

6. Обоснование порога отказа

Балл за режим кандидатов равен `10% × (0.7 × F1 + 0.3 × TNR)`. Оптимизировать надо именно эту комбинацию: составляющие тянут в разные стороны. Низкий порог отвечает на всё и обнуляет TNR, высокий отказывает на всём и обнуляет F1.

`falcon/calibrate.py` перебирает пороги по всему диапазону оценок и выбирает максимум комбинации на локальном сплите с долей open-set запросов около 20%. Метки закрытого теста не используются ни в каком виде.

Отчёт калибровки включает две базовые стратегии для сравнения — «всегда отвечать» и «всегда отказывать» — чтобы на защите было видно, что именно даёт выбранный порог.

Выбранный порог: 0.69

Три стратегии на 96 машинах, которых модели сдачи не видели (226 запросов, из них 51 без пары в галерее — 22.6%, близко к 20% закрытого теста). Балл посчитан эталонным скриптом организаторов:

Стратегия	F1	TNR	Балл за режим кандидатов
Всегда отвечать топ-1	0.8380	0.0000	0.5866
Всегда отказывать	0.0000	1.0000	0.3000
Порог 0.69	0.9174	0.9608	**0.9304**

Наивная стратегия «всегда выдавать лучшего кандидата» обнуляет TNR и в сумме проигрывает 3.4 балла из 10. Это ровно та ловушка, которую организаторы заложили в формулу `0.7 × F1 + 0.3 × TNR`.

Порог для моделей сдачи. Модели сдачи дообучены на 1445 машинах, и калибровать им порог на машинах, которые они видели, нельзя. Поэтому порог посчитан тем же способом, что описан ниже, но на оставшихся 96 машинах: четыре жеребьёвки × две плотности галереи, прореженная — 0.676 снимка на запрос, как в публичном тесте (`calibrate_threshold.py --extra-train-share 0.75 --test-gallery-per-query 0.676`). Максимум среднего — **0.69**, плато 0.65–0.72 (средний балл 0.898–0.909), собственные оптимумы прогонов 0.605–0.695. Шкала сходства у новых моделей выше: при прежнем пороге 0.605 средний балл был бы 0.869, а TNR упал бы до 0.73. На настоящих кадрах публичного теста, где пара есть у каждого запроса, при 0.69 сервис отказывает в 3.5% запросов — меньше, чем прежние модели при своём пороге (5.4%).

Ниже — как этот способ калибровки появился; числа в нём относятся к прежним моделям и прежней проверке из 385 машин.

Как выбран порог. Максимум на одном прогоне — плохой выбор: он подгоняется сразу под две случайности.

Жеребьёвка. `scripts/calibrate_threshold.py` делит отложенную часть четырьмя разными способами (обучающие машины те же, меняется только то, кто попал в галерею, кто в запросы и кто остался без пары — `partition_seed`). Собственные оптимумы разбиений расходятся: 0.595, 0.61, 0.675, 0.71.

Плотность галереи. У нас на 890 запросов 1541 снимок галереи, а в публичном тесте на 1110 запросов — всего 750. Чем реже галерея, тем ниже сходство с лучшим кандидатом и тем ниже оптимальный порог. Поэтому каждая жеребьёвка считается ещё и на прореженной до 750 снимков галерее, причём доля запросов без пары возвращается к 20% (иначе прореживание само плодит запросы без пары и завышает вес TNR). На редкой галерее оптимумы уходят к 0.535–0.605.

Плотность галереи	Собственный оптимум	Балл в нём	Балл при 0.605
наша, 1541 снимок	0.650	0.8549	0.8462
как в тесте, 750 снимков	0.580	0.7651	0.7618
среднее двух	0.605	—	**0.8040**

Сдаётся максимум среднего по обеим плотностям (со сглаживанием окном ±0.01, шаг сетки 0.005) — **0.605**. На закрытом тесте будет ровно одна жеребьёвка и одна плотность, и какие — неизвестно, поэтому выбор по среднему честнее выбора по лучшему: 0.605 теряет 0.006 на нашей галерее и выигрывает 0.023 на редкой по сравнению с порогом 0.64, подобранным только по плотной (0.804

против 0.796 в среднем). Кривые и выбор — на картинке docs/threshold_curves_1156.svg и в docs/threshold_choice_1156.json; для моделей сдачи — docs/threshold_curves.svg и docs/threshold_choice.json.

Проверка на настоящих тестовых кадрах: при 0.605 сервис отказывает на 5.4% запросов публичного теста, при 0.64 — на 6.7%, при 0.67 — на 8.5%. По ответу 17 организаторов у каждого запроса публичного теста пара есть, то есть каждый такой отказ — ошибка, и более низкий порог здесь тоже в плюс.

Порог калибруется заново при каждой смене модели: у ансамбля из двух ResNet он был 0.4925, у ансамбля с CLIP — 0.51, после PCA-проекции сходства стали контрастнее и порог ушёл к 0.6075, после калибровки по плотностям — 0.605, а у моделей, дообученных на 1445 машинах, — 0.69. Сама шкала при этом не менялась: уверенность всегда считается по исходному косинусу, а не по обогащённым векторам. Значение закреплено одной константой `CALIBRATED_THRESHOLD` в `falcon/submit.py`: её берут скрипт сдачи, образ и docker-compose.

Другие правила уверенности проверены тем же протоколом — восемь прогонов, у каждого правила свой порог по максимуму среднего (балл 0.7·F1 + 0.3·TNR):

Правило уверенности	Свой порог	Плотная	Редкая	Среднее
косинус (в сдаче)	0.605	0.8462	0.7618	**0.8040**
косинус − 0.5 · (сходство 2–4)	0.355	0.8519	0.7675	0.8097
косинус × (1 + отрыв от 2–4)	0.670	0.8523	0.7659	0.8091
косинус − 0.3 · (сходство 2-го)	0.435	0.8452	0.7675	0.8064
косинус обогащённого вектора	0.790	0.8467	0.7514	0.7990
отрыв первого от второго	0.070	0.7448	0.6484	0.6966
z-оценка по всей галерее	0.020	0.5865	0.5607	0.5736

Лучшее из них выигрывает 0.006 балла из 10 — 0.06 итогового балла. Взамен оно требует, чтобы на закрытом тесте совпала не только шкала сходства с верным кандидатом, но и «фон» галереи: правило вычитает сходство со 2–4 кандидатами, а оно зависит от того, сколько в галерее похожих машин. На публичном тесте эта разница видна прямо: медиана лучшего сходства там 0.94 против 0.75 у нас, а второго — 0.62. Ради 0.06 балла менять самую переносимую величину на менее переносимую не стоит, поэтому в сдаче остался простой косинус.

Переносить порог между моделями нельзя: положение максимума зависит от качества конкретной модели, поэтому он пересчитывается на каждом прогоне (`scripts/run_pipeline.py`, этап 4).

Двойники: третий ответ сервиса

Самая неприятная ошибка без номера — две разные машины одной модели в одинаковой оклейке. В данных такие есть: два фургона Lamoda (Lada Largus, ТС-925 и ТС-1022) одинаковы вплоть до наклейки с QR-кодом на задней двери; видимая разница — рисунок на крыше: у ТС-925 белый с красными пятнами, у ТС-1022 — в клетку. Кадр ТС-925 модель ставит ближе к чужому фургону (0.968), чем к своему (0.965), и порог тут не поможет: оба сходства намного выше него.

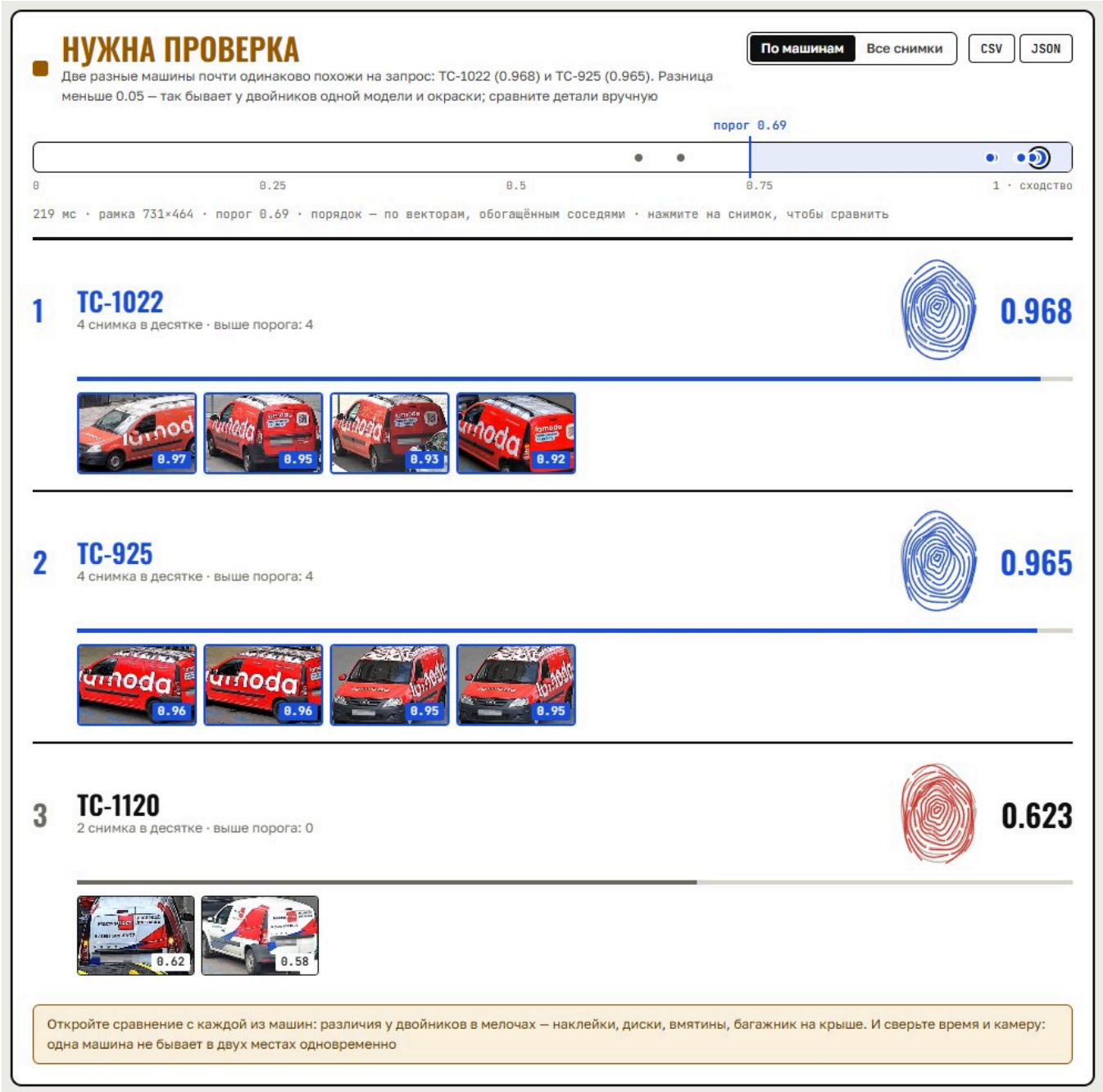
Угадать, какая из двух настоящая, сервис не может, но саму ситуацию он видит: над порогом две разные машины почти вровень. Тогда он не называет первую, а отвечает «требуется проверка» и

показывает обе, с подсказкой, на что смотреть при сравнении. Правило: у второй машины сходство не ниже порога и отстаёт от первой меньше чем на 0.05 (`find_twin` в falcon/calibrate.py, настройка `FALCON_TWIN_MARGIN`).

Зазор подобран на 96 машинах вне обучения моделей сдачи (scripts/twin_check.py, docs/twin_check.json). Из 151 принятого ответа 17 были ошибочными:

Зазор	Ошибок поймано	Верных ответов ушло на проверку	Всего на проверку
0.02	5 из 17	6 из 134	7%
0.03	7 из 17	10 из 134	11%
0.05	**10 из 17**	**15 из 134**	**17%**
0.08	11 из 17	18 из 134	19%
0.10	11 из 17	20 из 134	21%

При 0.05 доля ошибок среди уверенных ответов падает с 11% до 6%. Дальше выигрыш кончается: следующие 0.03 зазора ловят одну ошибку ценой трёх лишних проверок. На файлы сдачи правило не влияет: в формате организаторов у запроса либо ответ, либо отказ, третьего состояния нет. Это решение сервиса для оператора. В демо-галерее двойники есть (`scripts/seed_demo_gallery.py`, кадр-запрос `service/static/_test_twin.jpg`), и правило на остальных двенадцати машинах не срабатывает ни разу.



7. Внешние источники

Предобученные веса

Источник	Версия	Роль
ResNet50-IBN-a	релиз v1.0, `resnet50_ibn_a-d9d0bb7b.pth`	инициализация ResNet-ветви
CLIP ViT-B/16 (OpenAI, лицензия MIT)	`timm/vit_base_patch16_clip_224.openai` в Hugging Face Hub, загрузка через `timm==1.0.29`	инициализация CLIP-ветви

Предобученные веса используются только как инициализация и скачиваются один раз при обучении. В образ попадают наши дообученные чекпойнты; модель строится с `pretrained=False`, и при запуске

контейнер в сеть не ходит (`HF_HUB_OFFLINE=1`). Файлы зафиксированы по тегу релиза или идентификатору модели, как требует ответ 39.

Внешние датасеты

Датасет	Правообладатель	Получение	Использовано
VeRi-776	Xinchen Liu, BUPT	заявка автору, доступ 17.09.2026	`image_train`: 37 778 снимков, 576 TC, 20 камер

VeRi-776 служит промежуточным этапом обучения для `resnet-v2-veri.pt` и `clip-veri-ours.pt`: сначала модель учится на нём, затем дообучается на конкурсных данных. Условия и подробности — docs/EXTERNAL_DATA.md.

Библиотеки

Все версии зафиксированы точно в requirements.txt. Ключевые: `torch==2.14.0+cu126`, `torchvision==0.29.0`, `numpy==2.5.2`, `pillow==12.3.0`, `fastapi==0.141.1`, `starlette==1.6.0`, `uvicorn==0.53.0`, `pydantic==2.13.5`, `psycopg==3.2.3`, `pgvector==0.3.6`, `faiss-cpu==1.15.1`, `swagger-ui-bundle==1.1.0`, `timm==1.0.29`. Контейнеры: `nvidia/cuda:12.6.3-cudnn-runtime-ubuntu24.04`, `pgvector/pgvector:pg16`, `nginxinc/nginx-unprivileged:1.29-alpine`. Проверка известных уязвимостей: `pip-audit -r requirements.txt` — 0.

---

8. Структура репозитория

```
falcon/            конкурсное ядро
  metrics.py       официальный протокол оценки
  data.py          чтение CSV, кроп, PK-сэмплер, локальный сплит
  model.py         ResNet50-IBN-a + GeM + BNNeck; CLIP ViT-B/16 + BNNeck
  losses.py        ID-loss + batch-hard triplet + center
  transforms.py    препроцессинг и аугментации
  train.py         цикл обучения с продолжением после обрыва
  extract.py       экстрактор, ансамбль, замер производительности
  explain.py       Grad-CAM (ResNet и ViT) и проверка зоны номера
  rerank.py        k-reciprocal, совместимый с потоковым протоколом
  calibrate.py     подбор порога отказа
  submit.py        формирование и самопроверка файлов сдачи
service/           backend инференса
  app.py           FastAPI, OpenAPI, маршруты
  security.py      лимиты, заголовки, ключ API
  repository.py    pgvector и SQLite
  schemas.py       контракты API
  static/          интерфейс оператора (city.js — схема города)
web/               клиентская часть: nginx, интерфейс и прокси
models/            веса ансамбля из сдачи (Git LFS): 2 × CLIP и ResNet в fp16, PCA-проекция
submission/        файлы сдачи для публичного теста
scripts/           обучение, сдача, разбор ошибок, замеры
  overnight_clip.py    обучение CLIP и перебор ансамблей
  compare_ensembles.py выбор состава: точность, бутстреп, порог, скорость
  plate_masking_check.py  mAP@10 при закрашенной зоне номера
  finetune_plate_erase.py дообучение с закраской зоны номера
  export_weights.py    экспорт весов в fp16
  fit_projection.py    PCA-проекция вектора ансамбля по обучающей части
  calibrate_threshold.py  порог отказа по жеребьёвкам и плотностям галереи
  compare_release.py   сдача против нового кандидата: вердикт по всем критериям
  verify_official.py   сверка с эталонным скриптом организаторов
tests/             153 теста
docs/              разбор ошибок, безопасность, правовые вопросы, масштабируемость, отчёты
legacy/            архив ранних версий v1–v4 (не участвует в сдаче, см. legacy/README.md)
```

---

9. Узкие места: что может сломаться у жюри и что с этим сделано

Проверены не только метрики, но и то, что может помешать решению отработать на чужой машине. Каждый пункт найден проверкой, а не предположением.

Узкое место	Чем грозит	Что сделано
Процессор жюри — Xeon Gold 6338, 2.0 ГГц	задержка на кадр выше лимита 40 мс	Замерено: из ~30 мс на кадр 12.1 мс (медиана) уходит на процессор — чтение JPEG и масштаб, остальное — видеокарта. На медленном ядре процессорная часть вырастет примерно до 20 мс, зато A5000 примерно вдвое быстрее RTX 3060 Laptop: итог около 30 мс. Чтение JPEG уже идёт с понижением разрешения на этапе DCT (`load_crop`, `draft`)
Квота трафика Git LFS	вместо весов придут текстовые указатели	`scripts/fetch_weights.py` докачивает веса со сверкой SHA-256, сервис при указателях объясняет, что делать

Узкое место	Чем грозит	Что сделано
Большая закрытая галерея	обогащение векторов строило матрицу «галерея × галерея»: 50 000 снимков — 10 ГБ	Расчёт блоками по 2048 строк, память не зависит от размера галереи, результат совпадает (тест)
Контейнер `api` ещё не поднялся	nginx падал целиком, интерфейс пропадал	Имя `api` разрешается во время запроса: интерфейс открывается всегда и сам сообщает, что сервис не отвечает. Проверено остановкой `api`
Распределение закрытого теста иное	порог отказа, подогнанный под одну выборку, промахивается	Порог выбран по восьми прогонам: четыре жеребьёвки × две плотности галереи (раздел 6)
Драйвер жюри CUDA 12.2, в образе CUDA 12.6	CUDA не стартует	Minor version compatibility внутри CUDA 12; колёса PyTorch содержат машинный код для Ampere (sm_86), компиляция на лету не нужна
Первый запрос после старта	подбор алгоритмов cuDNN — секунды вместо миллисекунд	Прогрев при старте сервиса на батчах 1 и 16, вся работа с видеокартой — в одном прогретом потоке
Размер образа — 17 ГБ	долгая сборка и много места на диске у жюри	Разобрано: около 5 ГБ — копия CUDA и cuDNN из базового образа `nvidia/cuda`, которые PyTorch не использует (у него свои). Переход на базу `ubuntu:24.04` сокращает образ примерно вдвое, но на машине разработки его не удалось проверить: Docker Desktop под Windows падает при распаковке PyTorch, когда файл подкачки WSL переполняет диск C:. Непроверенную базу в сдачу не ставим

10. Ограничения и честные оговорки

- Все метрики измерены на **локальном** сплите конкурсного `train.csv`, а не на закрытом тесте. Протокол воспроизводит официальный, включая junk-фильтр и долю open-set запросов, но совпадения чисел с закрытым тестом это не гарантирует.
- Выборка для итоговых чисел небольшая: 96 машин, 175 запросов с парой и 51 без пары. Бутстрэп по машинам (снимки одной машины перетасовываются вместе) даёт 95% интервал 0.869–0.955 для mAP@10 и 0.892–0.963 для балла режима кандидатов (scripts/confidence_intervals.py, docs/confidence_intervals.json). Поэтому смену моделей решали по парному бутстрэпу разницы на тех же запросах, а не по точечным значениям. Порог и зазор правила двойников выбраны на тех же машинах, так что числа режима отказа скорее оптимистичны; порог держится на плато 0.65–0.72.
- Эталонный скрипт организаторов (`organizers/evaluate.py`) используется для всех итоговых чисел; наш `falcon/metrics.py` совпадает с ним до знака.
- Для ResNet обучение на 140 эпох вместо 60 не дало значимого прироста (лучший ансамбль из ResNet 0.6145 против 0.6119). Прирост дала смена архитектуры: CLIP ViT-B/16.
- В публичном тесте у запросов почти всегда есть очень похожий снимок в галерее (медиана лучшего сходства 0.94 против 0.75 на локальном сплите): судя по почти совпадающим рамкам, это соседние кадры той же камеры. Второй кандидат там уже далеко — медиана 0.62, — поэтому сервис отказывает лишь в 3.5% запросов против 31% на локальном сплите. Кадры той же

камеры с той же машиной эталонный скрипт убирает из ранжирования, а в режиме кандидатов засчитывает как верный ответ. Номеров камер в тестовых CSV нет, и выдача их не отсеивает.

- Обогащение галереи соседями (DBA) применяется только в пакетном режиме, где галерея известна целиком: в живом сервисе снимки добавляются по одному, и пересчитывать соседей всей галереи на каждую вставку нельзя. Запрос сервис обогащает так же, как пакетный режим (alpha-QE по сотне ближайших снимков галереи), и кандидаты сортируются по обогащённому вектору. Шкала уверенности общая: и там, и там это исходный косинус, поэтому порог один. Цена на отложенных машинах: mAP@10 0.907 против 0.914 в пакетном режиме, балл режима кандидатов 0.905 против 0.930 — почти целиком из-за TNR (0.90 против 0.96: три лишних ложных совпадения на 51 запрос без пары, интервал широкий).
- Замеры масштабируемости сделаны на синтетических векторах с кластерной структурой, а не на эмбеддингах обученной модели — см. оговорки в docs/SCALABILITY.md.
- Опора на номер проверена закраской зоны номера на всех снимках локального сплита (раздел 5): закраска рамки номера стоит 0.017 mAP@10, столько же, сколько худшая контрольная зона той же площади (0.017). Интерфейс повторяет проверку для каждой пары.
- Характерные ошибки — смена ракурса, темнота, двойники (каршеринг одной модели и окраски) и перекрытие — разобраны с примерами и цифрами в docs/ERROR_ANALYSIS.md, в том числе на настоящих кадрах публичного теста: там у каждого запроса есть пара, и все 39 отказов (3.5%) — наши ошибки, почти всегда из-за перекрытой машины.